Who moves first? Allocating a contested network

Background

Mobility gets hard not when a route is long, but when several journeys want the same space at the same time. One city corridor at eight in the morning carries an ambulance, two bus lines, a paratransit trip that has to arrive before a dialysis appointment, a dozen delivery vans with time windows, and everyone else. Capacity is fixed: lanes, stops, curb space, signal time, a shared charging point. Every plan that serves one claim well serves another worse.

A plan that is optimal on total travel time may be the plan that strands the dialysis passenger. That is not a flaw in the solver; it is a decision the objective function made on someone's behalf, usually without anyone noticing. Which claims a model treats as inviolable, which it quietly trades away, and whether a planner can see that happening - that is the question of this challenge.

Your Concrete Challenge!

The mobility department of a mid-sized city is redesigning the morning peak on one of its busiest corridors. Every plan on the table serves someone better and someone else worse, and so far, nobody has been able to say exactly who. Next month, the city council will vote on a plan. Before that, the head of planning asks you: which plans are worth considering, who wins and who loses in each, and which one would you recommend?

Given a network and a set of competing claims on it, produce an allocation: which journeys get which capacity, at which time, on which route. Define what improvement means and for whom. Then show how the plan moves when the objective, the constraints or the computational representation change. Do not deliver one answer. Deliver a small set of defensible alternatives with their winners and losers made explicit. Expect the situation to change during the hackathon, for example a new claim on the corridor: your approach should be able to adapt when it does.

Choose your quantum path

Allocation under exclusive capacity and hard priority is constrained combinatorial optimization: in binary variables it is an integer program, and with the constraints folded into the objective it becomes a QUBO, the native input of annealers and of QAOA. That much is textbook. Three questions are not, and they are the ones worth a hackathon:

- **Constraint handling.** An emergency vehicle's priority and a guaranteed paratransit ride are not preferences. In a penalty formulation they become numbers that a better objective value can outweigh. Which constraint-handling choice - penalty weights, constraint-preserving mixers, a hybrid decomposition - keeps an inviolable claim inviolable, and how do you prove it did?
- **Solutions as a distribution.** An exact solver returns one optimum. Quantum and quantum-inspired samplers return a distribution over solutions. For a decision with contested objectives, a ranked set of feasible allocations that differ in who benefits is more useful to a human planner than a single optimum. Test whether that is true here.
- **Scale, if you get that far.** Simulate what fits in the time you have. If you want to say something about the full instance, estimate what a real device would need - but a careful small result beats a rushed large one. Quantum-inspired methods are legitimate entries throughout.

Questions that may help you find your way:

- **Who is on the road?** Which claims compete for the corridor, and which of them may never be traded away?
- **What does better mean, and for whom?** Total travel time, the worst-off journey, the share of guaranteed rides served: which measure do you choose, and who does it favour?
- **Does the priority survive?** When an inviolable claim becomes a penalty term, can a better objective value buy it off?
- **One plan or several?** Is a ranked set of alternatives more useful to the planner than a single optimum?
- **Who needs it?** The head of planning, the dispatcher, the passenger waiting for the paratransit ride: what does each of them need to see?
- **Where does quantum help?** Where, if anywhere, does a quantum or quantum-inspired formulation let you think or compute differently?

Starting points

In A Framework to Formulate Pathfinding Problems for Quantum Computing, Robert Wille and his team show how conceptual requirements - visiting specified locations, respecting an order of visits, keeping paths separate - can be translated automatically into optimization models. Their MQT QUBOMaker software (since July 2026 part of MQT ProblemSolver: pip install mqt.problemsolver) supports interchangeable one-hot, domain-wall and binary encodings, and the paper shows that the encoding needing the fewest variables differs from case to case.

Contested allocation is that framework under load. Several actors move on one network at once, so "keeping paths separate" stops being a modelling nicety and becomes the mechanism by which capacity is shared. Add an order requirement (pickup before delivery, connection before departure), a required stop (the guaranteed ride) and a priority claim (the ambulance), and the model carries the entire political content of the plan in its constraint set.

This is where the challenge earns its depth: an inviolable claim has to survive the encoding**.** Teams can use QUBOMaker to generate several formulations of the same scenario, compare them by variable count and by solution quality, and then check what the comparison usually hides - whether the encoding preserved the priorities that were meant to be non-negotiable, or turned them into a price.

A second toolchain: Qamomile

Qamomile (JIJ) works at the other end of the same pipeline. It connects mathematical models — QUBO and Ising formulations — to quantum algorithms such as QAOA and VQE, and transpiles one program to several backends, among them Qiskit, CUDA-Q, QURI Parts and qBraid. Where QUBOMaker is strong on getting from requirements to a formulation, Qamomile is strong on carrying one formulation across execution stacks.

Offering both means teams can compare toolchains rather than follow a single prescribed pipeline, and a team that models the same scenario in both and reports where the two diverge has produced a result of its own.

Starting data and preparation

Time is short, so nobody should spend the first day on data plumbing. Organisers supply a ready-to-run package: a small cut-out network from OpenStreetMap for one corridor in Zurich, the Swiss public transport timetable in GTFS format (opentransportdata.swiss) for the scheduled claims, curated demand for the flexible ones, a loader notebook, and one reference model that already runs end to end. Two or three common scenarios make results comparable across teams: a morning peak, a disruption (a blocked lane or a failed line), and optionally an event day. Teams may change the objective, the network subset and the scale, and say why. A larger instance (for example VBZ passenger counts and planned versus actual running times from Open Data Zürich) is available for teams that want it, and no one is rewarded for using it.

Pre-learning: graphs and routing, demand and assignment, competing objectives and fairness measures, constraint modelling and encodings. No local transport knowledge is required.

Tutorial and additional material

One tutorial is part of the programme and everyone attends it together:

- **TUT-73 - Quantum Software Stacks: Overview, Requirements & Experiences (M. Schulz, Wille, Echavarria, Burgholzer, IEEE Quantum Week 2026).** How quantum methods connect to workflows that actually run - the background this challenge builds on.

The following are additional material. They are there if a team wants to go further in a particular direction, and nobody is expected to have worked through them:

- **TUT-71 - Practical Integration of Cloud-based Quantum Resources via QDMI into HPC–QC Stacks (Hopf, Burgholzer, Wille, Stern, Quetschlich, Takeshita).** For running on cloud or HPC resources instead of a local simulator.
- **TUT-140 - From Circuits to Results: Systematic Experiment Tracking in Quantum Workflows.** For teams that want to track and compare many runs systematically.
- **TUT-87 - MLIR for Quantum-Classical Compilation (Stade, Burgholzer, Reumann, Haag, Rovara, Hopf, Wille).** Deep toolchain material, for the curious.

MQT QUBOMaker and Qamomile come with worked examples; starting from one of those is a legitimate way to spend the first hours.

What remains open

Teams choose their formulation, their method and their fairness measure - worst-off journey, maximum regret, share of guaranteed claims served, or one they define and defend. MQT QUBOMaker and Qamomile are available resources, not a required pipeline.

You are not asked to prove anything against a classical solver. Benchmarking quantum against classical methods is a research programme, not a hackathon task, and forcing it would push teams into a comparison nobody can do properly in the time. The comparison we do ask for is inside your own approach: run at least two variants - two encodings, two ways of handling a hard constraint, or two objective weightings - and show what changes between them, especially for whom. If a classical or quantum-inspired baseline genuinely helps your argument, use it; it is welcome, not required. No claim of quantum advantage is expected or rewarded, and a well-supported finding that the distributional output adds nothing a planner can use is a valid outcome.

Responsibility and sustainability

Work with your GTPP policy advisers, who use the policy domain sheet for this challenge, and choose one responsibility or sustainability point. Build it into your work, not only into your responsibility note: it should change something you can show in your model or prototype, and you should be able to say what it changed. Possible points for this challenge: keep accessible and paratransit trips protected and check that this survives the encoding; add a criterion that favours public transport or lower emissions and show who pays for it; let affected users see and contest a plan that disadvantages them; or report the computing resources your quantum or quantum-inspired workflow needs compared with a classical one.

Expected deliverables

Every team builds a case for the head of planning and presents it in a short pitch. The city decides; your work should inform that decision, not replace it. You can choose between two formats:

- **Format A: Make the case.** Build the case and a prototype that visualises it, for example a map or timeline that shows the head of planning how each plan serves the competing claims and who pays the price.
- **Format B: Run the experiment.** Build a reproducible computational experiment that compares encodings, ways of handling a hard constraint or objective weightings, and show what changes between them, especially for whom.

Whichever format you choose, submit the following:

- **A defensible problem.** The claims on the network, which are inviolable and why, what improvement means and for whom, the requirements retained and the assumptions introduced.
- **A working model.** Code that runs on the supplied instance, and one visualisation of the allocation that a non-specialist can read.
- **A decision portfolio.** Two or three feasible plans rather than one, what separates them, and who gains or loses in each - in a form a planner could actually choose from.
- **A short findings note.** What your two variants changed, whether inviolable claims were ever traded away, where the approach broke down, and what you would do with another week. Two pages are plenty.
- **Decision log.** Which variants you tried and discarded, and why, including where you chose not to follow an AI tool's suggestion. AI tools are allowed; say where and how you used them.

- **Responsibility note.** What your plans support and what they leave out or flatten, whose claims your model could not represent, and what would be needed before the city could rely on it.
- **Pitch.** A five-minute presentation to the head of planning: which plan you recommend, why, who pays the price, whether the city could act on it tomorrow, what remains uncertain, and the strongest argument against your recommendation.
- **Decision Card.** One printed card (A5) that you hand to the decision-maker at the end of your pitch: your recommendation in one sentence, who wins and who loses, how confident you are and why, the one condition under which your recommendation should not be followed, and what an AI tool suggested and what you did differently. After your pitch, the jury draws a "what if" card, and you say live whether your recommendation still holds.

Assessment rewards meaningful modelling decisions, feasible solutions and clear explanations of trade-offs. Completing a workflow is not enough: the result has to establish something about the plan it produces and about who it serves. A small, honest, well-explained result outranks a large, unexamined one. Answers that could fit any plan score low. What counts is how specifically you work with the supplied network, the competing claims and your own results, and how well you defend your choices live in the pitch.